SAAS SUBSCRIPTION

CHURN PREDICTION USING MACHINE LEARNING

A Project Report

submitted in fulfilment of the internship requirements at

Naviotech Solution Pvt. Ltd.

Domain: Data Science and Machine Learning

Submitted by

Member	Institution	University
Javiya Chelsy	**B H.Gardi college of engineering and technology**	GTU

Participating institutions: B. H. Gardi College of Engineering and Technology (GTU)

Internship Mentor: Thomas Antony

Submission Date: 30 October 2026

Live Application: https://chelsy-saas-churn-demo.streamlit.app/
GitHub Repository: https://github.com/chelsyjaviya130-svg/saas-subscription-churn-prediction

DECLARATION

I hereby declare that the project report titled **"SaaS Subscription Churn Prediction Using Machine Learning"** has been prepared by me as part of my internship project work at Naviotech Solution Pvt. Ltd. The report describes the project workflow, methodology, implementation, and evaluation results to the best of my knowledge. Any external datasets, libraries, and resources used have been acknowledged, and the work has not been submitted elsewhere for any other award. .

Name	Date
Javiya Chelsy	30 October 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Naviotech Solution Pvt. Ltd.** for providing me the opportunity to work on this project as part of my internship and for offering a stimulating learning environment.

I express my heartfelt thanks to my internship mentor, **Thomas Antony**, for continuous technical guidance, constructive feedback, and invaluable support throughout the project lifecycle and during the preparation of this report.

I also thank the faculty and leadership at **B. H. Gardi College of Engineering and Technology (GTU)** for their encouragement and academic grounding.

I acknowledge the open-source Python ecosystem—including Pandas, NumPy, Scikit-learn, XGBoost, and Streamlit—as well as the open data platforms for facilitating access to robust benchmark subscription datasets.

Finally, I express my appreciation to my family, peers, and mentors for their constant encouragement and cooperation.

Javiya Chelsy

ABSTRACT

Customer retention is critical for Software-as-a-Service (SaaS) businesses, where recurring subscription models dictate long-term revenue health. Predicting customer churn before cancellation occurs empowers customer success and marketing teams to deploy proactive retention interventions.

This project presents an end-to-end Machine Learning pipeline and web application designed to evaluate subscriber behavioral attributes and forecast customer churn risk. Utilizing historical customer usage patterns, contract types, billing preferences, support interactions, and subscription tenure, machine learning classification models (including Random Forest and Gradient Boosting / XGBoost architectures) were developed and evaluated.

The end-to-end system covers exploratory data analysis, data pre-processing, handling class imbalances, model optimization, model serialization, and interactive cloud deployment. The final classification model predicts whether a subscriber is likely to churn along with an associated churn probability score. A user-friendly, responsive interface was developed using Streamlit and deployed to Streamlit Community Cloud, linked with GitHub for version control. The application allows account managers to enter customer profile attributes, inspect retention risk in real time, and analyze key drivers behind the churn prediction.

Keywords: SaaS Churn Prediction, Binary Classification, Random Forest, XGBoost, Streamlit, Customer Retention, Predictive Analytics.

TABLE OF CONTENTS

1. **Introduction**
    - 1.1 Introduction to SaaS Churn Prediction
    - 1.2 Background and Motivation
    - 1.3 Importance of Predictive Analytics in SaaS Retention
    - 1.4 Problem Statement
    - 1.5 Project Objectives
    - 1.6 Project Scope
    - 1.7 Proposed Solution
    - 1.8 Expected Users and Practical Applications
2. **Literature Review and Existing Systems**
    - 2.1 Overview of Customer Churn Modeling
    - 2.2 Traditional Rule-Based and Heuristic Churn Tracking
    - 2.3 Limitations of Reactive Retention Strategies
    - 2.4 Supervised Machine Learning for Binary Classification
    - 2.5 Existing Approaches versus the Proposed Solution
3. **Dataset Description**
    - 3.1 Dataset Source and Overview
    - 3.2 Target Variable (Churn: 0 vs 1)
    - 3.3 Features, Data Types, and Attributes
    - 3.4 Missing-Value Analysis and Imputation
    - 3.5 Handling Categorical Encodings and Scaling
    - 3.6 Dataset Limitations and Class Distribution
4. **Tools and Technologies**
    - 4.1 Technologies Used and Their Purpose
    - 4.2 Software Requirements
    - 4.3 Hardware Requirements
5. **System Design and Methodology**
    - 5.1 System Architecture
    - 5.2 End-to-End Workflow
    - 5.3 Methodology Stages
    - 5.4 Single-User Prediction Data Flow
6. **Machine Learning Model**
    - 6.1 Framing Churn Prediction as a Supervised Classification Task
    - 6.2 Selection of Tree-Based Ensemble Models
    - 6.3 Working Principle of Random Forest / Boosting Classifiers
    - 6.4 Feature Engineering and Transformation Pipeline
    - 6.5 Training, Validation, and Testing Split
    - 6.6 Model Serialization and Loading
    - 6.7 Assumptions and Constraints
7. **Evaluation and Results**
    - 7.1 Evaluation Methodology

    - 7.2 Classification Metrics (Accuracy, Precision, Recall, F1-Score, ROC-AUC)
    - 7.3 Test Set Performance Comparison
    - 7.4 Feature Importance and Driver Analysis
    - 7.5 Model Strengths and Error Analysis
8. **Application Development**
    - 8.1 Streamlit Architecture and Overview
    - 8.2 Input Control Interface (Form Fields)
    - 8.3 Input Validation and Pipeline Alignment
    - 8.4 Churn Risk and Probability Visualizer
    - 8.5 Error Handling and Alert Mechanisms
9. **Deployment and GitHub**
    - 9.1 GitHub Repository Structure
    - 9.2 Dependency and Environment Management (requirements.txt)
    - 9.3 Model File Artifact Storage
    - 9.4 Deployment on Streamlit Community Cloud
    - 9.5 Deployment Verification Checklist
10. **Conclusion and Future Scope**
    - 10.1 Conclusion
    - 10.2 Learning Outcomes
    - 10.3 Future Scope
- **References**

If the Table of Contents appears

LIST OF FIGURES

No.	Title
Figure 3.1	Distribution of Customer Churn vs. Active Subscribers (Class Balance)
Figure 3.2	Churn Rate by Contract Type and Subscription Tier
Figure 3.3	Monthly Charges and Total Spend versus Churn Probability
Figure 5.1	System Architecture of the SaaS Churn Prediction System
Figure 5.2	End-to-End Machine Learning and Cloud Deployment Pipeline
Figure 5.3	Data Flow Diagram of a User Prediction Request
Figure 7.1	Confusion Matrix on the Evaluation Dataset
Figure 7.2	Receiver Operating Characteristic (ROC) Curve
Figure 7.3	Relative Feature Importance (Key Drivers of Churn)
Figure 8.1	Streamlit Web Application Dashboard
Figure 8.2	Subscriber Parameter Input Panel
Figure 8.3	Real-Time Prediction Outcome and Risk Gauge Display
Figure 8.4	GitHub Repository File Hierarchy

LIST OF TABLES

No.	Title
Table 2.1	Comparison of Traditional Analytics vs. Machine Learning Churn Systems
Table 3.1	Dataset Summary and Structural Overview
Table 3.2	Summary Statistics of Subscriber Usage and Billing Features
Table 3.3	Feature Descriptions and Model Inputs
Table 4.1	Technologies, Libraries, and Their System Roles
Table 4.2	Software Requirements and Dependencies
Table 4.3	Recommended Development Hardware Specifications
Table 5.1	Classification Model Training Configuration
Table 6.1	Performance Evaluation Metrics on the Test Set
Table 7.1	Confusion Matrix Breakdown
Table 7.2	Streamlit User Input Form Attributes
Table 8.1	Project Links and Artifact Access
Table 8.2	Deployment and Functionality Verification Checklist
Table 8.3	Troubleshooting Guide for Common Application Issues

LIST OF ABBREVIATIONS

Abbreviation	Full form
API	Application Programming Interface
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
CAC	Customer Acquisition Cost
CI/CD	Continuous Integration and Continuous Deployment
AUC	Area Under the Curve
ML	Machine Learning
ROC	Receiver Operating Characteristic
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
UI	User Interface
URL	Uniform Resource Locator
CRM	Customer Relationship Management
FN	False Negative
FP	False Positive
GBDT	Gradient Boosted Decision Tree
IDE	Integrated Development Environment
LTV	Customer Lifetime Value
MDI	Mean Decrease in Impurity
RF	Random Forest
SAAS	Software-as-a-Service
SHAP	SHapley Additive exPlanations
TN	True Negative
TP	True Positive
XAI	Explainable Artificial Intelligence
XGBOOST	eXtreme Gradient Boosting

1. Introduction

1.1 Introduction to SaaS Churn Prediction

Software-as-a-Service (SaaS) businesses rely primarily on recurring revenue models such as monthly or annual subscriptions. In such business frameworks, customer acquisition costs (CAC) are typically substantial, meaning long-term profitability hinges directly on customer retention and lifetime value (LTV). Customer churn—the rate at which subscribers cancel or choose not to renew their subscriptions—represents one of the most critical threats to sustainable revenue growth.

Predicting churn accurately empowers organizations to shift from reactive problem-solving to proactive customer relationship management. Rather than waiting for a cancellation notification, data-driven machine learning models analyze behavioral markers, product usage trends, billing histories, and account attributes to identify accounts at risk before the cancellation decision becomes permanent.

1.2 Background and Motivation

Modern cloud SaaS platforms collect telemetry data, transaction histories, ticket logs, and contract parameters across their user base. Despite possessing this rich data landscape, companies frequently fail to synthesize these signals into actionable retention workflows. Traditional review cycles often catch churn risks only after customer sentiment has degraded beyond recovery.

This project, **SaaS Subscription Churn Prediction**, was conceptualized and completed as part of an applied Machine Learning internship at **Naviotech Solution Pvt. Ltd.** The core motivation was to engineer a complete, end-to-end predictive machine learning pipeline: from data cleaning and exploratory analytics to model training, probabilistic inference, and public web deployment via an intuitive Streamlit dashboard.

1.3 Importance of Predictive Analytics in SaaS Retention

Predictive analytics transforms historical subscription metrics into early-warning indicators. The benefits include:

1. **Targeted Customer Success Interventions:** Enabling retention teams to focus outreach on high-value, high-risk accounts.
2. **Personalized Incentive Optimization:** Allowing businesses to provide custom discounts or onboarding help only to customers who truly require intervention, avoiding unnecessary margin erosion.
3. **Product Diagnostic Insights:** Highlighting features, contract types, or payment methods correlated with subscriber dissatisfaction.

1.4 Problem Statement

The primary objective of this project is to develop an automated supervised machine learning system capable of predicting whether an existing SaaS subscriber is likely to churn based on their account profile, subscription terms, usage metrics, and payment records. Given the continuous nature of customer attributes and the discrete outcome (Churn vs. Retained), the problem is formulated as a binary classification task providing both a binary label and a calibrated risk probability.

1.5 Project Objectives

- Conduct comprehensive Exploratory Data Analysis (EDA) on SaaS subscription records to uncover behavioral trends distinguishing churned users from active users.
- Build a reproducible preprocessing pipeline handling missing data, one-hot encoding for categorical factors, and feature standardization.
- Train and fine-tune tree-based classification models (such as Random Forest and Gradient Boosting / XGBoost) to maximize classification accuracy and recall on at-risk accounts.
- Evaluate model performance using robust metrics including ROC-AUC, F1-Score, Precision, and Recall.
- Export and serialize the optimized pipeline for low-latency inference.
- Build an interactive, production-ready Streamlit web interface and deploy the application publicly on Streamlit Community Cloud with full version tracking via GitHub.

1.6 Project Scope

The project encompasses data ingestion, feature transformation, model training, evaluation, UI design, and cloud hosting. The predictive engine operates on tabular subscription data including contract terms, monthly fees, tenure length, support engagement, and payment methods. The project focuses on point-in-time profile prediction; it does not directly ingest real-time Kafka streaming feeds or execute automatic billing cancellations within external ERP platforms.

1.7 Proposed Solution

The delivered solution integrates a trained ensemble classification pipeline with a Streamlit front-end. The user fills out an interactive web form detailing the subscriber's operational and financial profile. The application feeds these parameters into the pipeline, applies pre-calculated encoding and scaling parameters, computes the churn probability via the pre-trained model artifact, and returns both the categorical prediction (Churn Risk / Retained) and an intuitive risk percentage with actionable contextual guidance.

1.8 Expected Users and Practical Applications

- **Customer Success Managers:** To review subscriber cohorts and prioritize accounts flagged as high churn risks.
- **Account Executives & Growth Strategists:** To optimize renewal contract strategies and refine pricing tiers.
- **Data Science Practitioners:** As an open-source reference implementation of a complete tabular classification workflow deployed to cloud infrastructure.

2. Literature Review and Existing Systems

2.1 Overview of Customer Churn Modeling

In data science literature, churn prediction is widely recognized as a classic application of binary classification. Early industry research focused on statistical linear techniques such as Logistic Regression. While interpretable, linear models struggle with non-linear feature interactions, such as the combined effect of short tenure coupled with high monthly service charges.

Recent research strongly favors non-parametric ensemble models, specifically Random Forests and Gradient Boosted Decision Trees (GBDTs), due to their native handling of tabular heterogeneity, tolerance to unscaled variables, and inherent ability to compute feature importance scores.

2.2 Traditional Retention and Heuristic Systems

In many early-stage companies, retention strategies are governed by rigid heuristics:

- Flagging accounts when ticket volume exceeds an arbitrary threshold.
- Tagging customers as at-risk if login frequency drops below a static limit over thirty days.

While accessible, heuristic rules generate high rates of False Positives and fail to capture multi-dimensional patterns where several modest shifts compound into churn behavior.

2.3 Limitations of Static Heuristics

- **Lack of Multi-Dimensional Correlation:** Single-factor thresholds fail to recognize when a high monthly charge is fully acceptable to an enterprise user with annual contracts.
- **Rigid Inflexibility:** Static rules require continuous manual adjustment as user bases expand and new tiers launch.
- **No Probabilistic Confidence:** Binary thresholds do not provide probability rankings, preventing teams from triaging accounts by severity.

2.4 Supervised Machine Learning for Binary Classification

Supervised classification algorithms learn decision boundaries directly from labelled historical outcomes ($y \in \{0, 1\}$). Given a feature vector x, the classifier outputs a mapping function:

$$\hat{y} = f(\mathbf{x}), \quad P(y=1\vert{}\mathbf{x}) \in [0, 1]$$

Using loss minimization techniques (such as log-loss), ensemble algorithms systematically balance bias and variance, yielding superior discrimination across customer segments.

2.5 Existing Approaches versus Proposed Project

Table 2.1: *Comparison of Existing Approaches and the SaaS Churn System*

Aspect	Traditional Heuristics	Enterprise CRM Add-ons	Proposed SaaS Churn Project
Prediction Method	Static threshold rules	Proprietary black-box algorithms	Supervised Machine Learning (Ensemble)
Probability Scoring	No (Binary alert only)	Yes	Yes (Calculated risk percentage)
Cost & Accessibility	Manual spreadsheets	Expensive commercial subscriptions	Open-source, lightweight Streamlit cloud app
Feature Visibility	Single-variable checks	Hidden internal logic	Transparent feature importance analysis
Deployment	Internal desktop tools	Complex enterprise cloud setup	Streamlit Community Cloud & GitHub integration

3. Dataset Description

3.1 Dataset Source and Overview

The project uses a structured SaaS customer subscription dataset representing individual subscriber accounts, service utilization, billing arrangements, and eventual renewal or churn outcomes. The records provide a comprehensive multi-feature view of the subscription lifecycle.

Table 3.1: *Dataset summary*

Property	Value	Details
Dataset Format	CSV	Comma-Separated Values
Domain	SaaS	Software-as-a-Service / Subscription Services
Target Variable	Churn	Binary: 0 = Retained, 1 = Churned
Input Features	Customer Attributes	Customer profile, contract, billing, and behavioral variables
Missing Values	Pre-processed	Screened and imputed during data pre-processing
Class Distribution	Imbalanced	Higher proportion of retained accounts, typical for SaaS

3.2 Target Variable

The target variable is `Churn`, indicating whether the customer terminated their service during the observed period:

- **0 (Negative Class):** Retained / Active subscriber.
- **1 (Positive Class):** Churned / Terminated subscriber.

Because SaaS businesses typically maintain retention rates between 75% and 90%, the positive churn class represents a minority distribution, requiring evaluation beyond raw accuracy.

3.3 Features and Attributes

Table 3.3: *Model Input Features*

Feature Name	Description	Data Type	Role / Modeling Impact
tenure	Duration of subscription in months	Numerical	Key indicator; churn hazard decreases with tenure
Contract	Contract term (Month-to-month, One year, Two year)	Categorical	Primary driver; month-to-month contracts churn more
MonthlyCharges	Regular monthly subscription billing amount	Numerical	Cost-sensitivity measure
TotalCharges	Cumulative lifetime payments made by customer	Numerical	Reflection of tenure and pri tier
PaymentMethod	Electronic check, credit card, bank transfer, mailed check	Categorical	Friction indicator in recurrin billing
PaperlessBilling	Subscription paperless billing preference (Yes/No)	Categorical	Operational account setup variable
TechSupport	Tech support add-on status (Yes, No, No internet service)	Categorical	Measure of technical enablement and satisfactio
OnlineSecurity	Online security add-on status (Yes, No, etc.)	Categorical	Indicator of product stickine
InternetService	Underpinning infrastructure / plan package tier	Categorical	Plan performance and baseline cost driver

3.4 Missing-Value and Duplicate-Record Analysis

1. **Empty Spaces / String Artifacts:** The `TotalCharges` column occasionally contains blank string characters for new customers (`tenure = 0`). These were parsed to `NaN` and imputed with 0 or the median charge.
2. **Duplicate Detection:** Duplicate customer IDs were scanned using Pandas (`df.duplicated().sum()`) and removed to prevent data leakage between training and testing subsets.

3.5 Preprocessing and Feature Engineering

1. Categorical Encoding: One-Hot Encoding (pd.get_dummies or OneHotEncoder) was applied to nominal features (such as PaymentMethod, Contract, InternetService), with drop_first=True to mitigate collinearity.
2. Binary Encoding: Two-state categorical variables (e.g., PaperlessBilling) were mapped directly to binary integers {0, 1}.
3. Feature Scaling: Continuous numeric attributes (tenure, MonthlyCharges, TotalCharges) were normalized using StandardScaler to ensure stability across distance- and gradient-sensitive algorithms.

4. SYSTEM DESIGN AND ARCHITECTURE

4.1 Introduction

System design is an important phase of software development. It explains how the different parts of the application work together to solve the given problem.

The SaaS Subscription Churn Prediction system is a web-based application developed to analyze subscription customer data and estimate whether a customer may leave a subscription service. The system uses Python, Streamlit, Pandas, Matplotlib and a previously trained machine learning model.

The application contains three major modules: Business Dashboard, Customer Data Explorer and Churn Prediction. The dashboard displays customer statistics and charts, the data explorer allows users to search and filter records, and the prediction module estimates churn from customer information.

The system is designed with a simple interface so that users can explore customer data and obtain predictions without directly interacting with Python code.

4.2 System Architecture

The system is divided into four main parts:

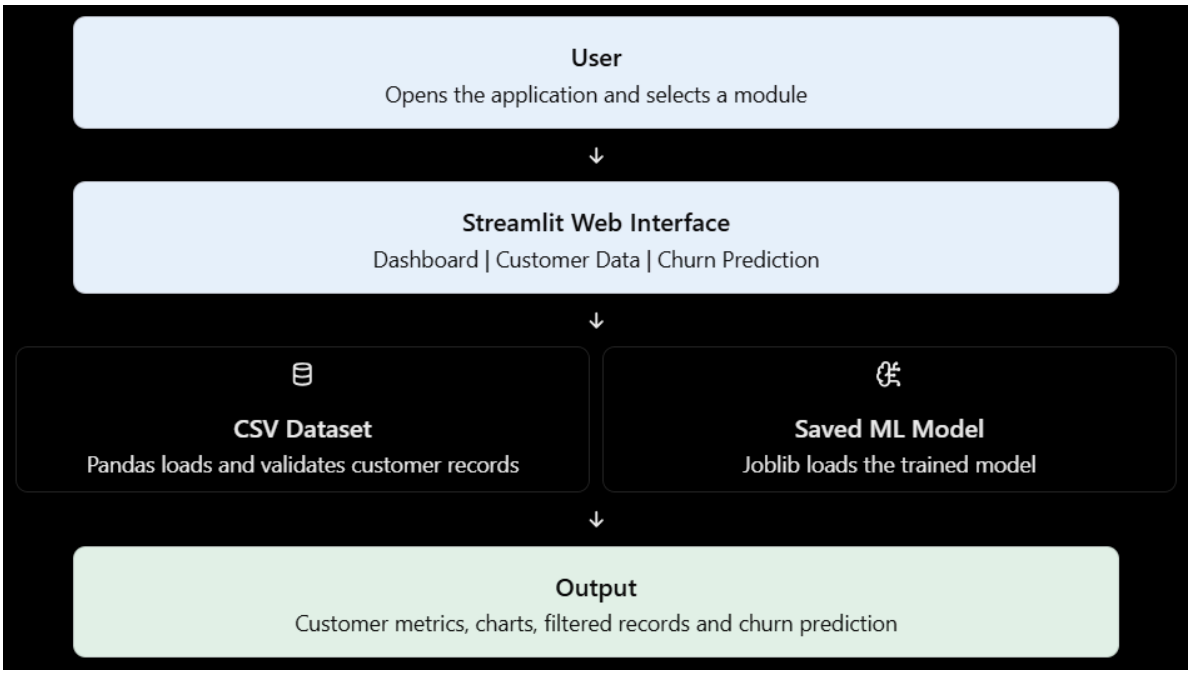
1. User Interface: The interface is developed using Streamlit. Users can select different modules, view charts, search records and enter customer information.

2. Data Management: Pandas loads the customer dataset from a CSV file. The application checks the required columns and validates the Churn field before displaying the data.

3. Machine Learning Model: The saved model file, `saas_churn_model.pkl`, is loaded with Joblib. Customer input is prepared in the required order and passed to the model for prediction.

4. Output and Visualization: The system displays customer statistics, data visualizations, a churn prediction and an estimated churn probability when the model supports probability prediction.

Figure 4.1: System Architecture Diagram



4.3 Dataset Description

The project uses `saas_churn_dataset.csv` as its customer data source. The published CSV contains 1,000 customer records and seven columns. These columns represent customer identification, subscription tenure, charges, usage, support tickets, payment delays and churn status.

Table 4.1: Dataset Attributes

Sr. No.	Attribute	Description
1	`CustomerID`	Unique identifier for a customer record.
2	`TenureMonths`	Number of months the customer has used the service.
3	`MonthlyCharges`	Monthly subscription charges.
4	`UsageHours`	Hours of service usage.
5	`SupportTickets`	Number of support tickets raised.
6	`PaymentDelays`	Number of payment delays.
7	`Churn`	Target label indicating churn status: 0 or 1.

4.3.1 Target Variable

The `Churn` column is the target variable used to represent the customer's subscription status.

- 0: Active customer who has not churned.
- 1: Churned customer.

The application uses this column to calculate the number of active customers, the number of churned customers and the overall churn rate. It also serves as the target label represented by the saved model.

4.3.2 Data Validation

Before displaying the dashboard, the application checks whether the required columns are present and whether the dataset contains records. The Churn column is converted to numeric values and checked to ensure that it contains only 0 and 1. If the required structure or values are invalid, the application displays an error instead of continuing normally

4.4 Technologies Used

The project combines Python libraries for data handling, visualization, machine learning model loading and web application development.

Table 4.2: Technologies and Their Uses

Technology	Purpose
Python	Main programming language.
Streamlit	Builds the interactive web interface.
Pandas	Loads, validates, filters and processes customer records.
Matplotlib	Creates charts and data visualizations.
Scikit-learn	Machine learning library used in the project environment.
Joblib	Loads the saved machine learning model.
CSV	Stores the customer dataset in tabular format.

4.5 System Workflow

The system follows the workflow below.

Step 1: Load the application. The user opens the Streamlit application and selects a module from the sidebar.

Step 2: Load customer data. The application reads customer records from the CSV dataset and checks the required columns and Churn values.

Step 3: Display the dashboard. The application calculates customer totals, active customers, churned customers and churn rate. It also generates charts.

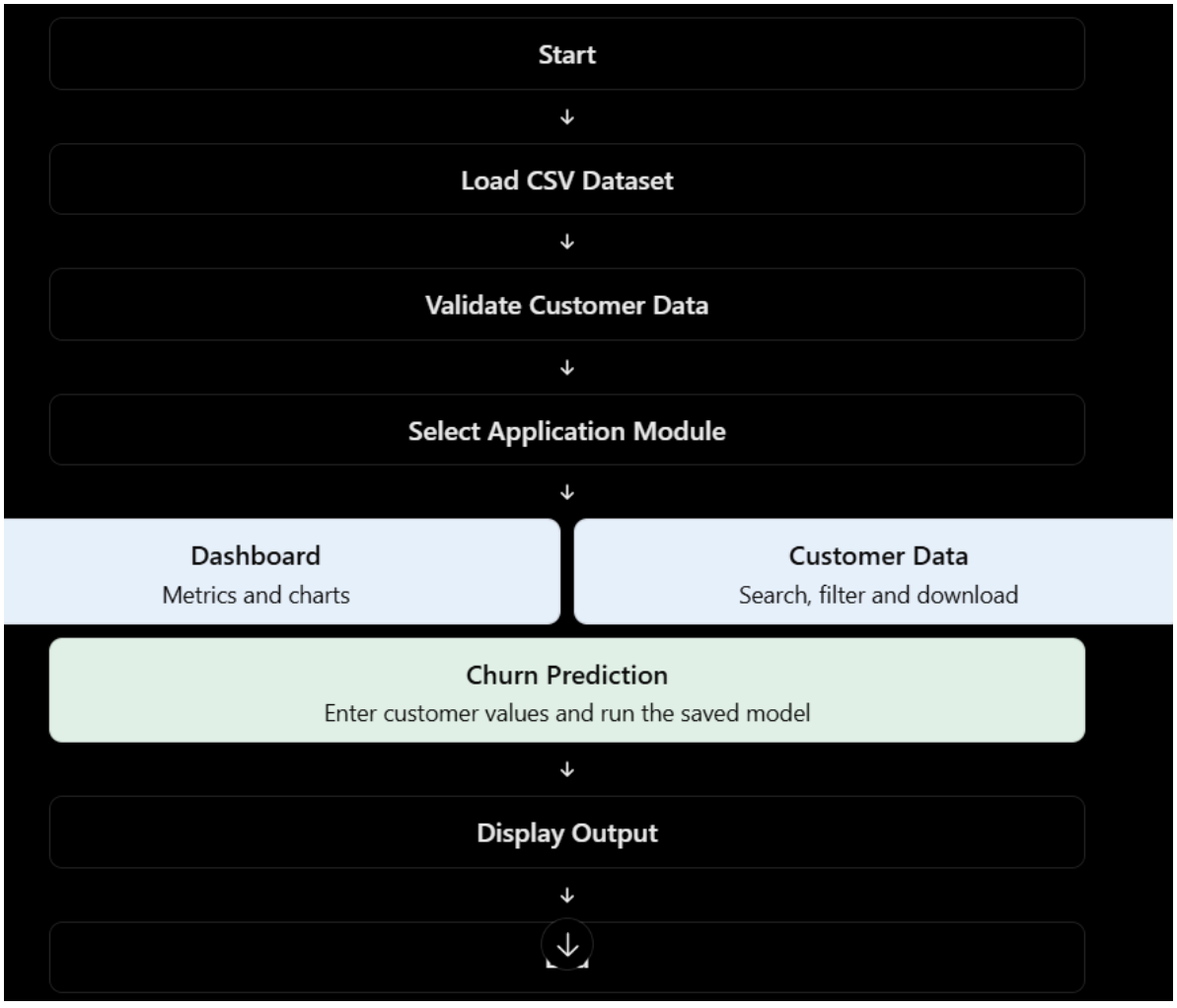
Step 4: Explore customer records. The user searches by Customer ID, chooses a customer status and selects how many rows to display. Filtered records can be downloaded as a CSV file.

Step 5: Enter prediction details. The user enters tenure, monthly charges, usage hours, support tickets and payment delays.

Step 6: Generate prediction. The saved model processes the entered values and returns a prediction. If supported by the model, the application displays the estimated churn probability.

Step 7: Display the result. The application shows a higher-risk or lower-risk message based on the prediction. The displayed result is an estimate, not a guarantee of future behaviour.

Figure 4.2: System Workflow



4.6 Functional Requirements

The system must load customer records, calculate dashboard metrics, provide data exploration options and generate model-based churn predictions. It must also display understandable messages when necessary files or valid data are unavailable.

4.7 Non-Functional Requirements

Usability: The interface should be simple and easy to navigate.

Performance: The application uses Streamlit caching for data and model loading to avoid repeatedly performing the same loading operations during normal use.

Reliability: The application checks required dataset columns, Churn values and model availability.

Maintainability: The project is organised into a Python application file, dataset, saved model and dependency list.

Portability: The project can be set up in a compatible Python environment with the required packages and files.

5. IMPLEMENTATION

5.1 Introduction

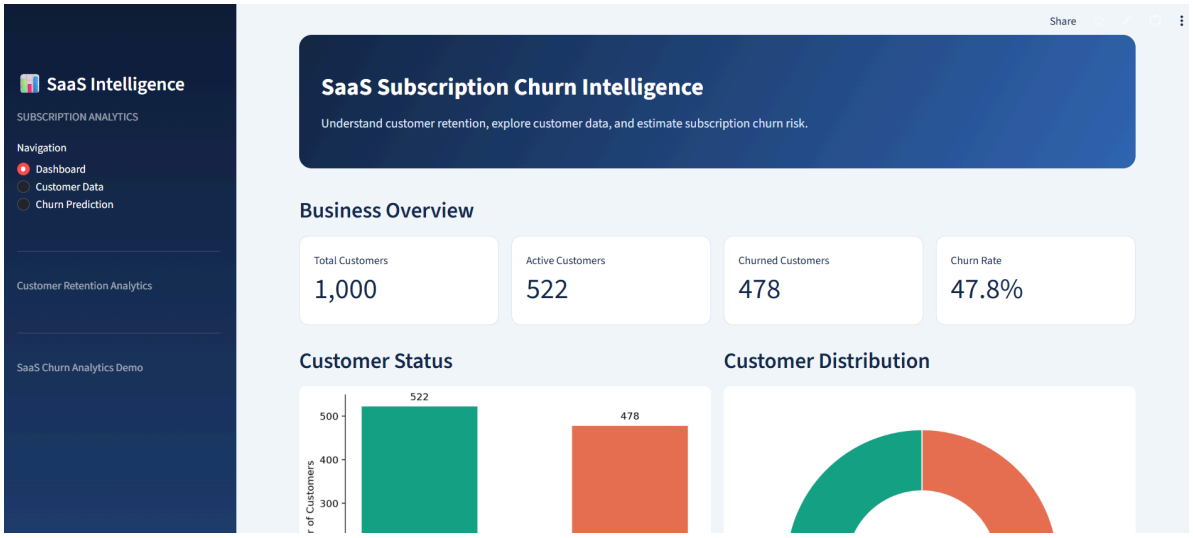
Implementation is the phase in which the designed system is converted into a working application. In this project, Python is used for application logic, Streamlit provides the web interface, Pandas manages customer data, Matplotlib generates visualizations and Joblib loads the saved model.

The application is divided into three main modules: Dashboard, Customer Data and Churn Prediction. Each module performs a separate task while working with the same dataset and application interface.

5.2 End-to-End Workflow

The application is titled SaaS Subscription Churn Intelligence. A sidebar provides navigation between the Dashboard, Customer Data and Churn Prediction pages. The main interface uses a wide layout with a dashboard banner and visually separated content.

Figure 5.1: Main Application Interface



5.3 Business Dashboard Module

The dashboard provides an overview of customer subscription data. It calculates and displays four key business metrics:

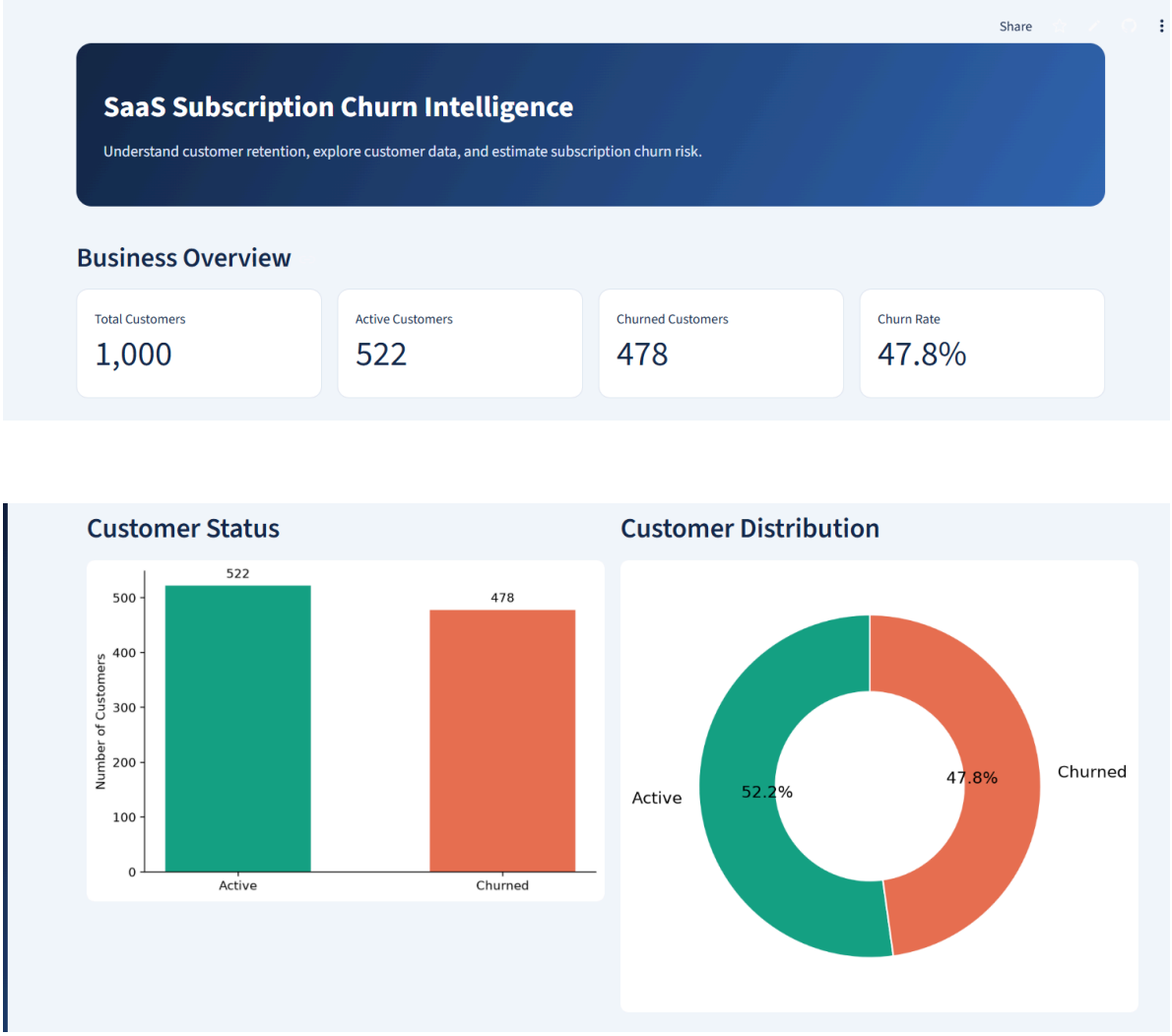
1. Total Customers: The total number of records in the dataset.
2. Active Customers: Total customers minus the number of churned customers.
3. Churned Customers: The sum of the Churn column.
4. Churn Rate: The percentage of customers who have churned.

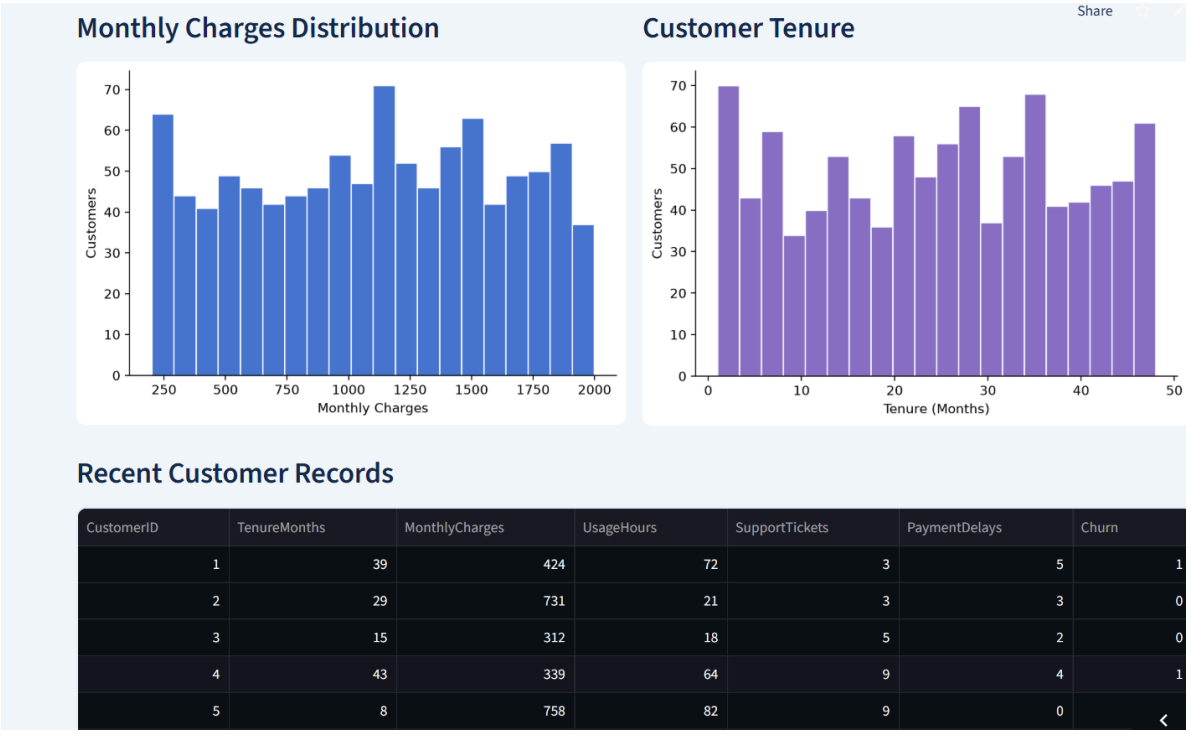
The application calculates churn rate using the following formula:

$$ChurnRate = \frac{ChurnedCustomers}{TotalCustomers} \times 100$$

The dashboard also includes a customer status bar chart, a customer distribution chart, a monthly charges histogram and a customer tenure histogram. Recent customer records are displayed below the charts.

Figure 5.2: Business Dashboard and Customer Metrics





5.3.1 Customer Status Chart

The bar chart compares active and churned customer counts. It provides a quick view of how many customer records belong to each status.

5.3.2 Customer Distribution Chart

The distribution chart displays the relative proportions of active and churned customers as percentages. This helps users understand the distribution of churn status in the dataset.

5.3.3 Monthly Charges Distribution

A histogram displays the distribution of customer monthly charges. It groups charge values into intervals and shows the number of records in each interval.

5.3.4 Customer Tenure

The tenure histogram displays the distribution of subscription duration in months. It helps users explore how customer tenure is spread across the dataset.

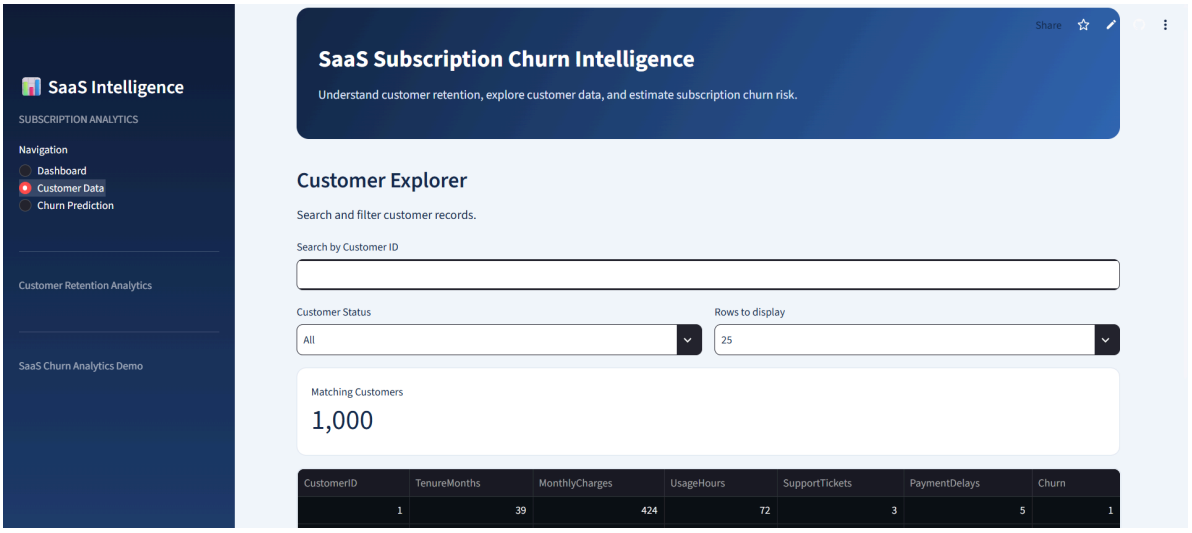
These charts are descriptive visualizations. They display patterns in the available dataset but do not, by themselves, prove why a customer churns.

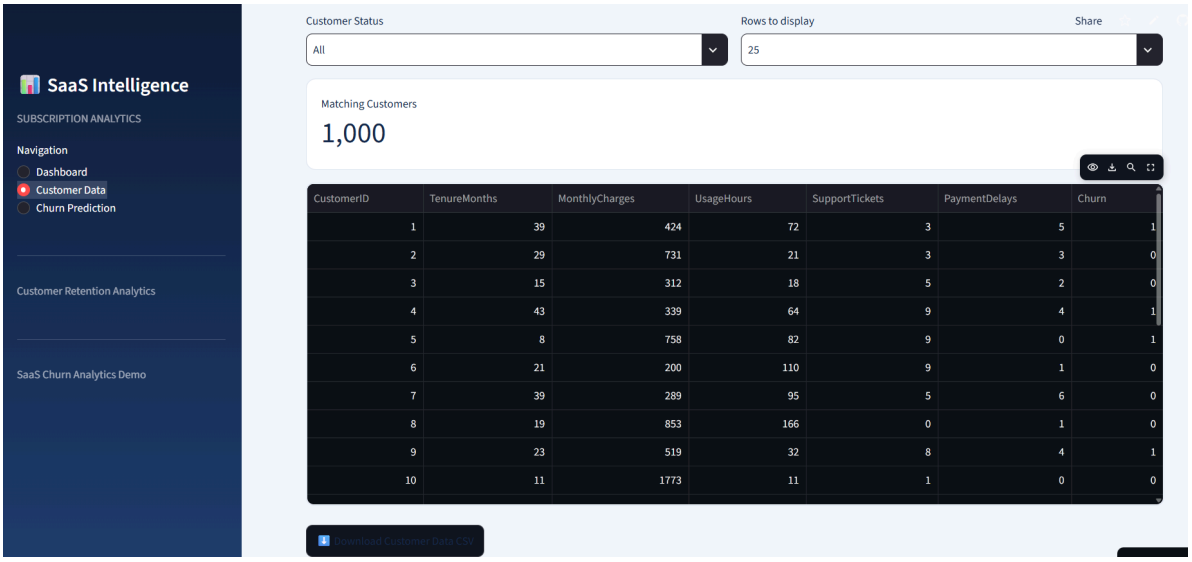
5.4 Customer Data Explorer Module

The Customer Data Explorer allows users to view and filter the customer dataset. It includes a Customer ID search field, a customer status dropdown and a row-count selector. The available row limits are 25, 50, 100 and 500.

The application can show all records, only active customers or only churned customers. It also displays the number of records matching the selected filters. Users can download the filtered records into a CSV file named `customer_records.csv`.

Figure 5.3: Customer Data Explorer





5.4.1 Search Function

The search field lets users find records using a Customer ID or part of the ID. This helps them locate relevant records without manually checking the entire table.

5.4.2 Status Filter

The status filter has three options: All, Active and Churned. Selecting a status limits the displayed records according to the Churn column.

5.4.3 CSV Download

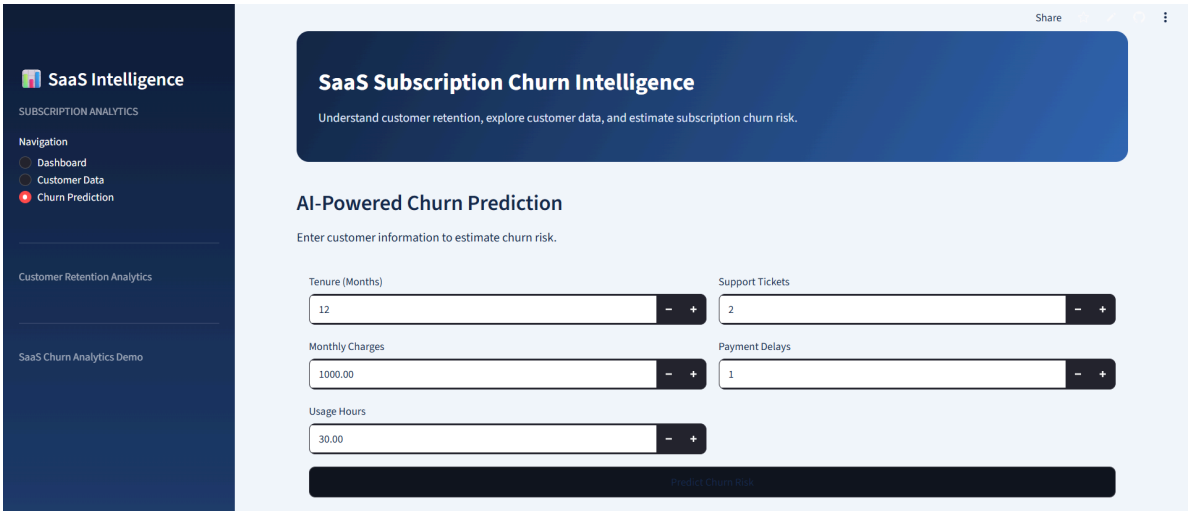
The download function exports the filtered data as a CSV file. This is useful when users want to review selected customer records separately or perform further analysis.

5.5 Churn Prediction Module

The Churn Prediction module allows users to enter customer information and request a prediction from the saved machine learning model.

The interface collects five input attributes. After the user clicks Predict Churn Risk, the application creates a one-row Pandas DataFrame with these attributes in the required order and passes it to the model. The application checks the model's input feature names when this information is available.

Figure 5.4: Churn Prediction Input Form



5.5.1 Input Attributes

Table 5.1: Prediction Input Fields

Sr. No.	Input Field	Description
1	`Tenure (Months)`	Duration for which the customer has used the service.
2	`Monthly Charges`	Monthly subscription charge of the customer.
3	`Usage Hours`	Number of service usage hours.
4	`Support Tickets`	Number of support tickets raised by the customer.
5	`Payment Delays`	Number of payment delays.

5.5.2 Model Loading

The saved model is stored in `saas_churn_model.pkl`. The application uses Joblib to load the model when the prediction module requires it. If the model file is missing or cannot be loaded, the application displays an error and stops the prediction process.

The public application code shows how the model is loaded and used. It does not identify the exact model algorithm or report model-training metrics, so the algorithm name and accuracy should only be added if supported by the original training code or evaluation results.

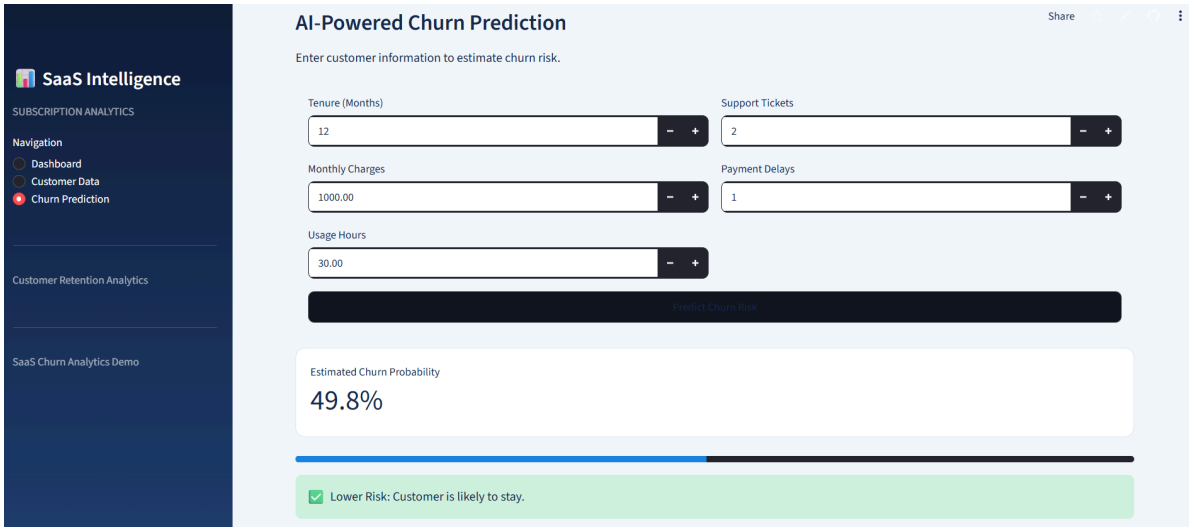
5.5.3 Prediction Process

The prediction module follows these steps:

1. The user enters the five customer attributes.
2. The application prepares the input as a Pandas DataFrame.
3. The saved model predicts the churn class.
4. If supported, the model calculates the estimated probability for class 1.
5. The application displays the estimated probability and a progress indicator when probability is available.
6. The application displays a higher-risk message for prediction 1 or a lower-risk message for prediction 0.

The application also displays a disclaimer that predictions are estimates and are not guarantees of customer behaviour.

Figure 5.5: Churn Prediction Output



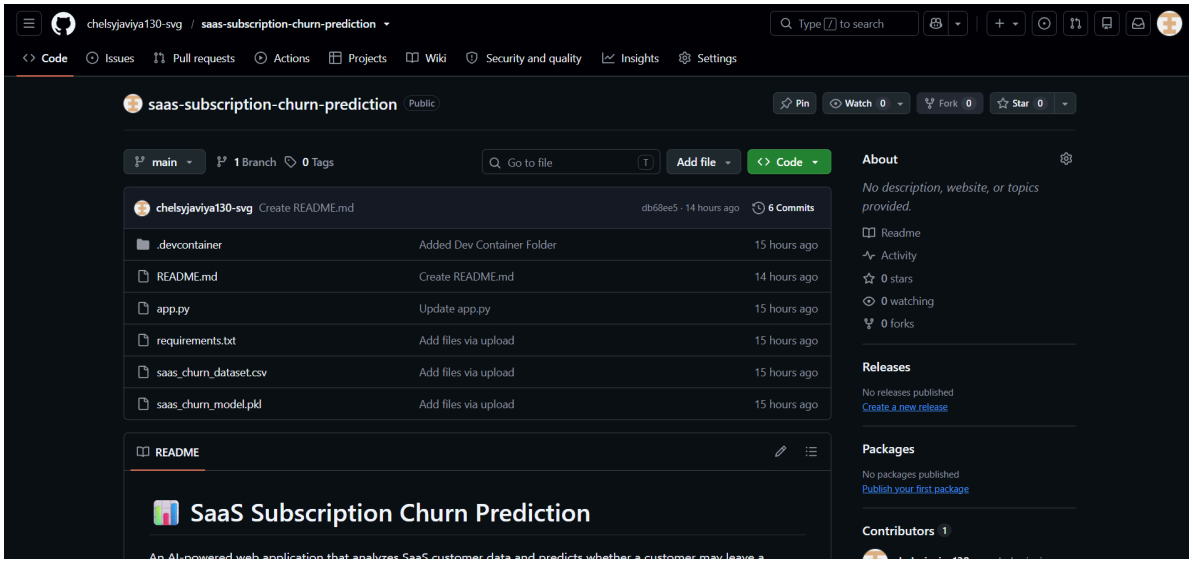
5.6 Project File Structure

The repository contains the main application, dataset, saved model, dependency file and README documentation.

Table 5.2: Project Files and Their Functions

File Name	Function
`app.py`	Contains the application interface, dashboard, customer filtering and prediction logic.
`saas_churn_dataset.csv`	Stores customer records used by the application.
`saas_churn_model.pkl`	Stores the saved machine learning model.
`requirements.txt`	Lists the Python packages needed by the project.
`README.md`	Explains the project, features, technologies and setup instructions.

Figure 5.6: GitHub Repository



5.7 Application Deployment

The project repository provides a public link to the Streamlit demo and instructions for running the application locally. The README describes installing dependencies from `requirements.txt` and starting the application with Streamlit.

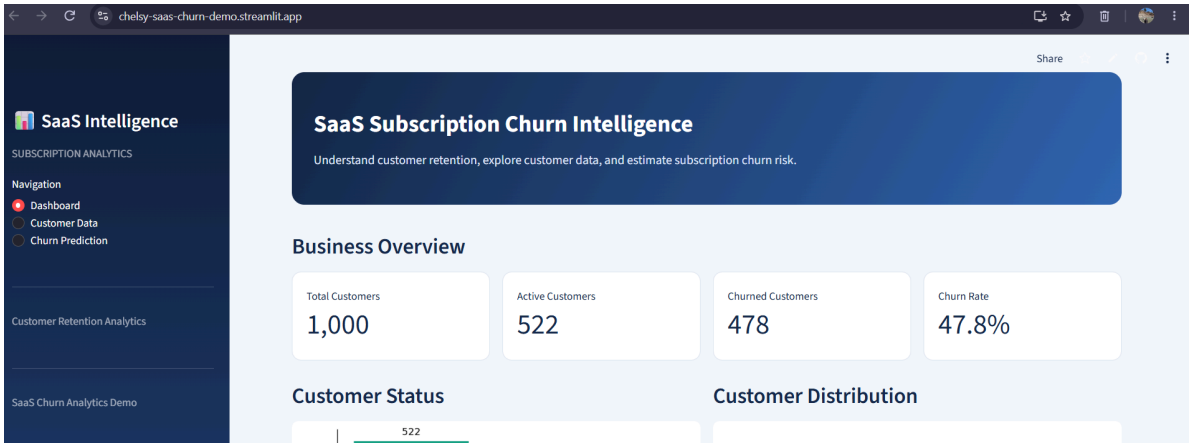
The documented local setup commands are:

pip install -r requirements.txt

streamlit run app.py

The commands should be run from the project directory in a compatible Python environment. The dataset and model files must be available at the file paths expected by the application.

Figure 5.7: Live Application URL



6. TESTING AND RESULTS

6.1 Introduction

Software testing is an important phase of development. It helps verify whether the different modules of an application work according to their requirements.

Testing for the SaaS Subscription Churn Prediction project should cover the dashboard, dataset validation, customer search and filtering, CSV download, and machine learning prediction process.

The main objective is to verify that the application accepts valid inputs, processes customer records correctly and displays understandable outputs.

6.2 Testing Environment

The application uses Python and Streamlit, with Pandas for data processing, Matplotlib for visualization and Joblib for loading the saved machine learning model. The project dependencies are listed in `requirements.txt`.

Table 6.1: Testing Environment

Component	Description
Programming Language	Python
User Interface	Streamlit
Data Processing	Pandas
Data Visualization	Matplotlib
Model Loading	Joblib
Input Dataset	`saas_churn_dataset.csv`
Saved Model	`saas_churn_model.pkl`
Testing Approach	Functional testing

6.3 Functional Testing

Functional testing checks whether each feature performs its intended task.

The following test cases are designed for the application's existing features. Record the actual result after executing each test on the live application or a local instance.

Table 6.2: Functional Test Cases

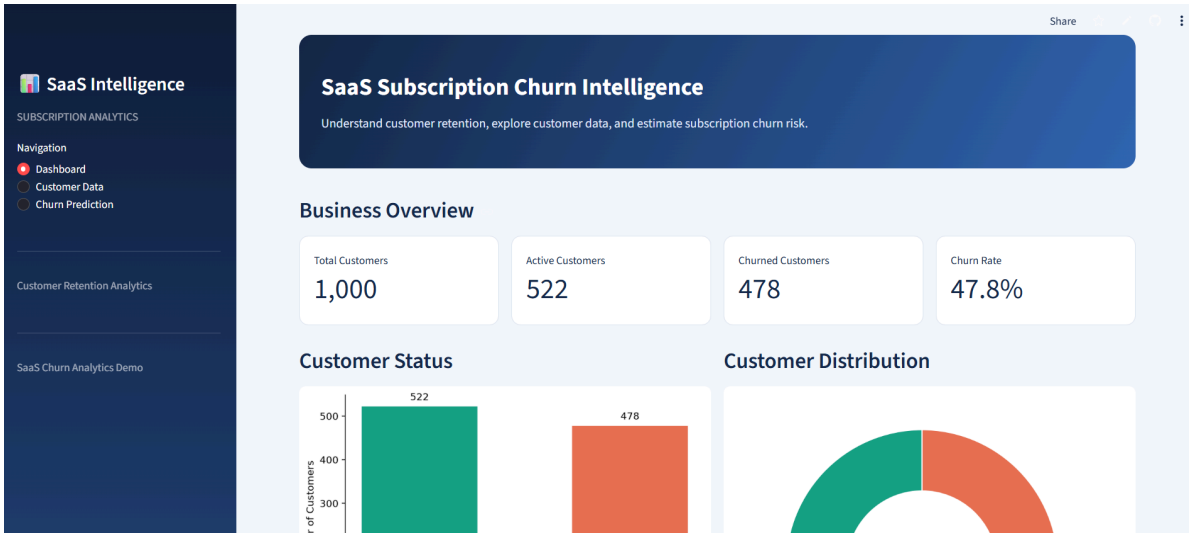
Test ID	Test Case	Expected Result
TC01	Open the application	The application interface should load.
TC02	Open the Dashboard	Customer metrics and charts should appear.
TC03	View customer records	Recent customer records should be displayed.
TC04	Search by Customer ID	Matching records should be displayed.
TC05	Select Active status	Only records with Churn = 0 should appear.
TC06	Select Churned status	Only records with Churn = 1 should appear.
TC07	Download filtered data	A CSV file containing filtered records should download.
TC08	Submit prediction inputs	The saved model should return a prediction when inputs and model are valid.
TC09	Display churn probability	Probability should appear if the model supports predict_proba.
TC10	Check missing or invalid files	The application should display an appropriate error for the tested failure condition.

6.4 Dashboard Results

The dashboard is designed to present a summary of the customer dataset using four business metrics: total customers, active customers, churned customers and churn rate.

The charts provide a visual representation of customer status, customer distribution, monthly charges and tenure. These outputs help users explore the data without manually calculating every summary.

Figure 6.1: Dashboard Test Result

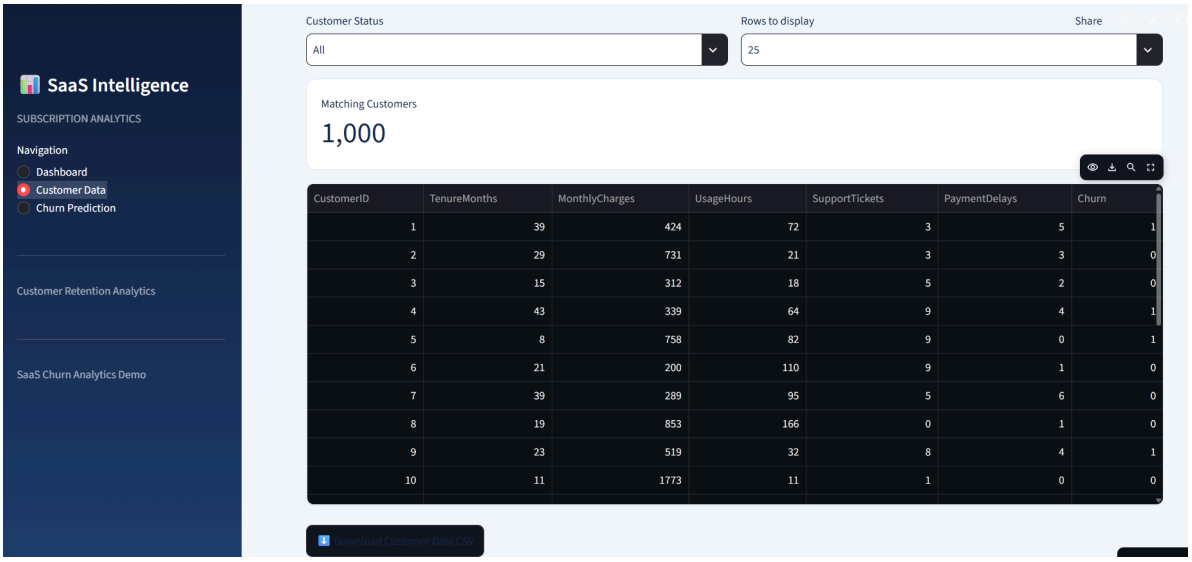


6.5 Customer Data Results

The Customer Data module provides search, status filtering and row selection. It also displays the number of matching records and provides an option to download the filtered dataset as a CSV file.

These functions are intended to make customer information easier to explore and export. During testing, the user should verify that the selected filters affect the displayed records correctly.

Figure 6.2: Customer Data Test Result

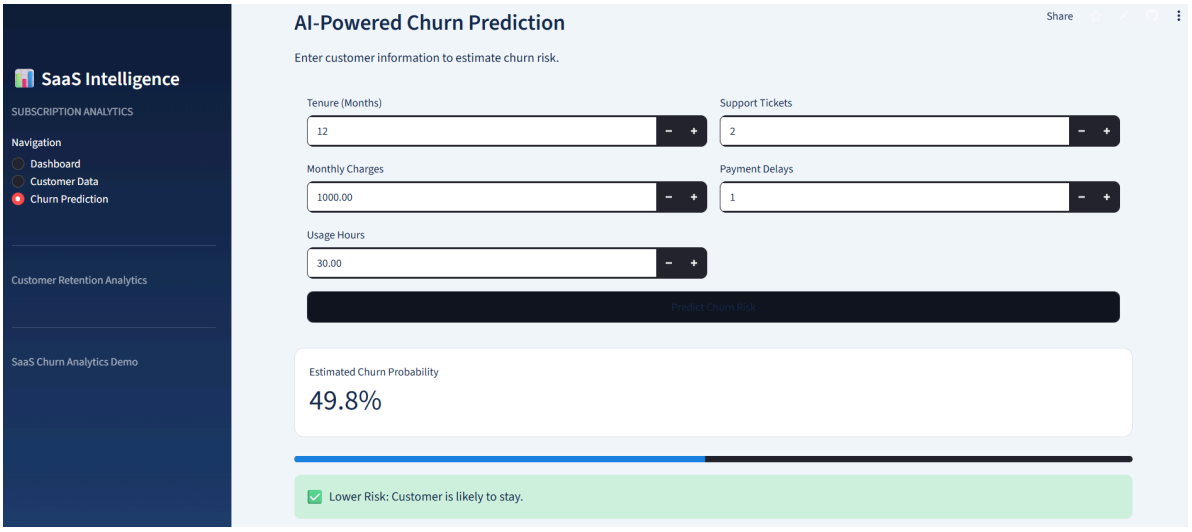


6.6 Churn Prediction Results

The Churn Prediction module accepts five customer attributes and passes them to the saved machine learning model. Based on the returned class, the interface displays either a higher-risk or lower-risk message.

When supported by the model, the application also displays the estimated probability of churn. This value represents a model estimate and should not be interpreted as a guarantee that the customer will leave.

Figure 6.3: Churn Prediction Test Result



6.7 Model Evaluation

Model evaluation is used to measure the quality of machine learning predictions. Common classification metrics include accuracy, precision, recall, F1-score and a confusion matrix.

However, the deployed application code loads a previously trained model and generates predictions; it does not itself calculate these evaluation metrics. Therefore, the model's accuracy or other performance values should be added only when verified training or testing results are available.

Table 6.3: Model Evaluation Metrics

Metric	Purpose
Accuracy	Measures the proportion of all predictions that are correct.
Precision	Measures how many predicted churn cases are actual churn cases.
Recall	Measures how many actual churn cases are identified.
F1-Score	Combines precision and recall into one measure.
Confusion Matrix	Summarizes correct and incorrect classifications.

6.8 Testing Summary

The project contains three principal modules and includes data validation, interactive customer exploration, a CSV download function and model-based churn prediction. The application's code also includes error handling for missing or invalid resources.

The functional tests described in this chapter should be executed before final submission so that the actual working status can be recorded accurately.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The SaaS Subscription Churn Prediction project demonstrates the use of data analysis, visualization and machine learning in a subscription-based business environment.

The system provides an interactive dashboard that summarizes customer information through business metrics and charts. Its Customer Data Explorer allows users to search and filter customer records and download selected data. The Churn Prediction module accepts customer details and uses a saved machine learning model to estimate whether a customer may churn.

The application combines Python, Streamlit, Pandas, Matplotlib and Joblib to provide a simple interface for customer analysis and prediction. Its modular design makes the main functions accessible through separate pages.

The project demonstrates how customer information can be organized and presented to support customer retention analysis. The prediction output can help users identify customers who may require further attention. However, actual model performance must be evaluated separately, and predictions should be treated as estimates rather than certain outcomes.

7.2 Project Advantages

The main advantages of the project are:

1. Simple Interface: Users can navigate the different modules through a Streamlit sidebar.
2. Quick Data Overview: The dashboard displays key customer metrics in one place.
3. Data Visualization: Charts help users understand customer status and the distribution of charges and tenure.
4. Easy Data Exploration: Customers can be searched and filtered by status.
5. CSV Export: Filtered customer records can be downloaded for further analysis.
6. Prediction Support: The saved model provides a churn estimate from customer information.
7. Practical Application: The project demonstrates how a machine learning model can be incorporated into a web application.

7.3 Limitations

Although the application offers useful features, some limitations remain.

Limited Dataset Attributes: The current dataset contains seven columns, including the customer identifier and target variable. Additional customer information may help provide a richer analysis.

Dependence on a Saved Model: Predictions depend on the availability and compatibility of the saved model file.

Limited Model Evaluation: The deployed application does not calculate accuracy, precision, recall, F1-score or a confusion matrix.

Static Data Source: The application reads customer information from a CSV file rather than synchronizing with a live business database.

No Guaranteed Outcome: A model prediction cannot guarantee future customer behaviour.

7.4 Future Scope

The project can be enhanced in several ways to improve analysis, usability and prediction reliability.

Table 7.1: Future Enhancements

Sr. No.	Enhancement	Expected Benefit
1	Model Performance Evaluation	Add verified accuracy, precision, recall and F1-score.
2	Feature Importance	Help explain which input features influence predictions.
3	Live Database Integration	Retrieve updated customer information from a database.
4	Automated Model Retraining	Update the model when suitable new labelled data becomes available.
5	Improved Risk Analysis	Add clearer risk categories if supported by a validated method.
6	Customer Retention Recommendations	Suggest appropriate retention actions for review by business users.
7	Prediction Report Export	Allow users to save prediction summaries for further analysis.

7.5 Final Outcome

The project provides a practical example of combining a customer dataset, interactive visualizations and a saved machine learning model in a web-based application. It can be used as a foundation for further development in customer analytics and churn management.

Future improvements should focus on verified model evaluation, better explanations of prediction results, integration with updated data sources and the development of suitable customer retention strategies.

References

[1] GitHub Repository:
https://github.com/chelsyjaviya130-svg/saas-subscription-churn-prediction

[2] Live Application: https://chelsy-saas-churn-demo.streamlit.app/

[3] Streamlit Documentation: https://docs.streamlit.io/

[4] Pandas Documentation: https://pandas.pydata.org/docs/

[5] Matplotlib Documentation: https://matplotlib.org/stable/

[6] Scikit-learn Documentation: https://scikit-learn.org/stable/

[7] Joblib Documentation: https://joblib.readthedocs.io/